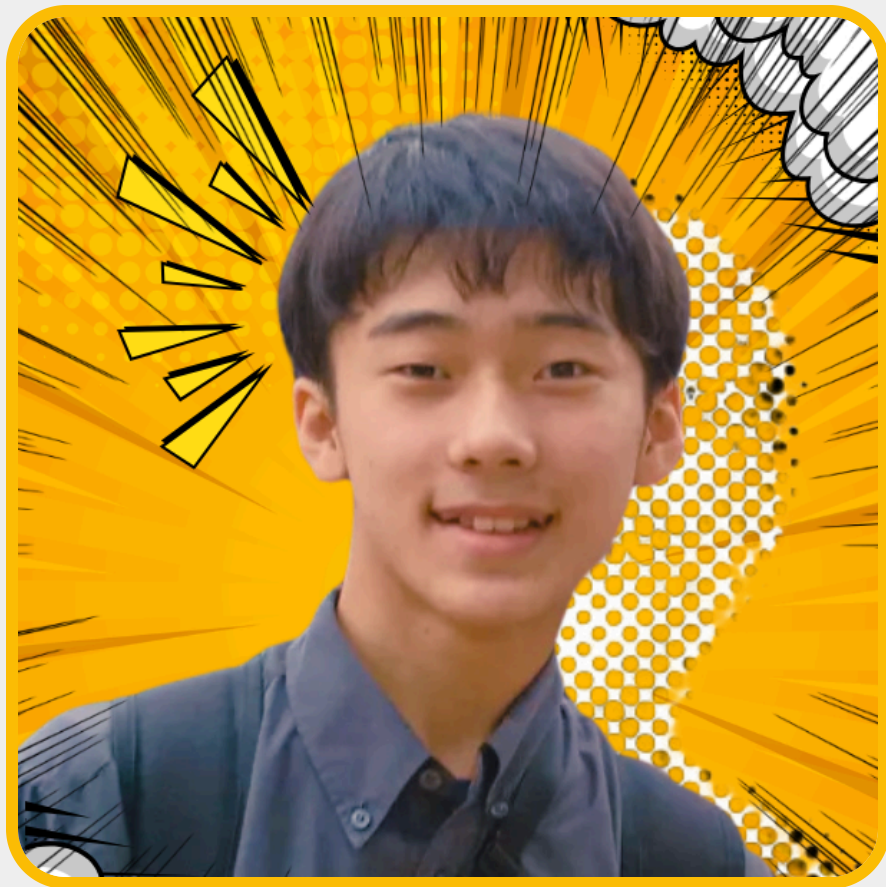


PHPの命名規則、結局 **camelCase** と **snake_case** どっちなんだ

PHPの歴史から、命名の層を見ていきます

田中博悠 / tanahiro2010

自己紹介



田中博悠 / tanahiro2010

- 三田学園高等学校 1年生
- GDG Greater Kwansai / Alpha+ Project
- 趣味: バンジー / 読書
- 最近:
まだ見ぬスカイダイビングに思いを馳せ中

PHPを書いているで思ったこと

```
str_replace()  
array_map()  
json_encode()  
file_get_contents()
```

でも最近のコードはこう見える

```
$request->getParsedBody();  
$response->getStatusCode();  
$userRepository->findById($id);
```

じゃあPHP、
どっちなん？

どっちが正しいか、だけでは終われません

PHPの命名規則が なぜ分かれて見えるのか

歴史から眺めて、明日からどう命名するかを考えます

先に私の結論

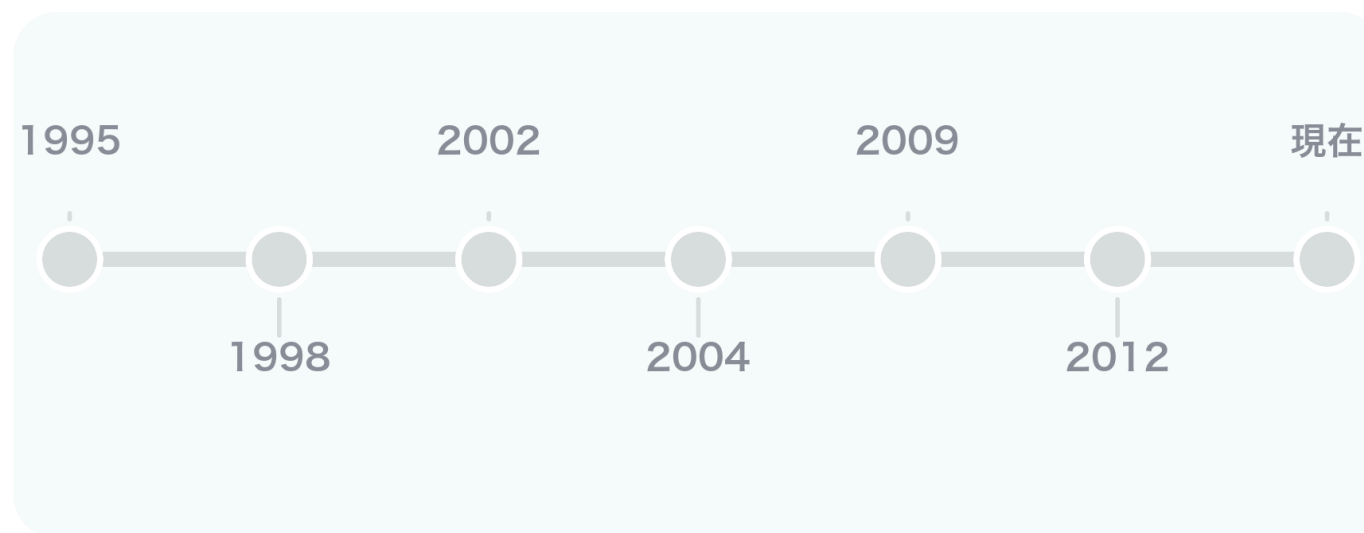
普通の関数

`snake_case` が自然そう

メソッド

`camelCase` が自然そう

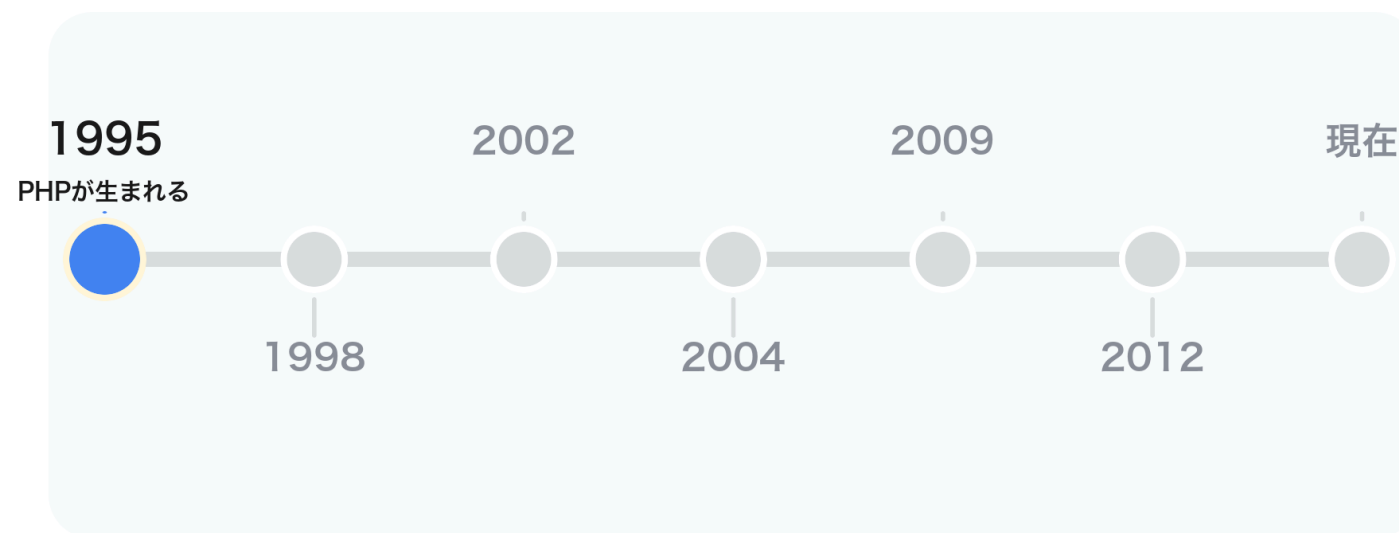
今日歩くPHP命名史



関数文化 → OOP文化 → PSR → 現代の判断

1995
PHPが生まれる

今日歩くPHP命名史



関数文化 → OOP文化 → PSR → 現代の判断

PHPは巨大言語として設計されたわけではない

Rasmus Lerdorf氏

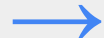
個人用ツール

Cで書かれたCGI

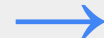
最初から30年後の巨大エコシステムを想定していた、という話ではありません

Personal Home Page Tools

オンライン履歴書



アクセス追跡



便利なツール群

ここでもず関数文化が育ちます

Webで便利な処理を 関数として呼び出す

文字列、配列、ファイル、フォーム、DB

標準関数、だいたいsnake_case

str_replace

array_map

json_encode

file_get_contents

mb_strlen

mysqli_connect

ただし完全には統一されていない

**「PHP標準関数 = すべてsnake_case」
ではありません**

でも、関数APIには`snake\_case`が多く見える

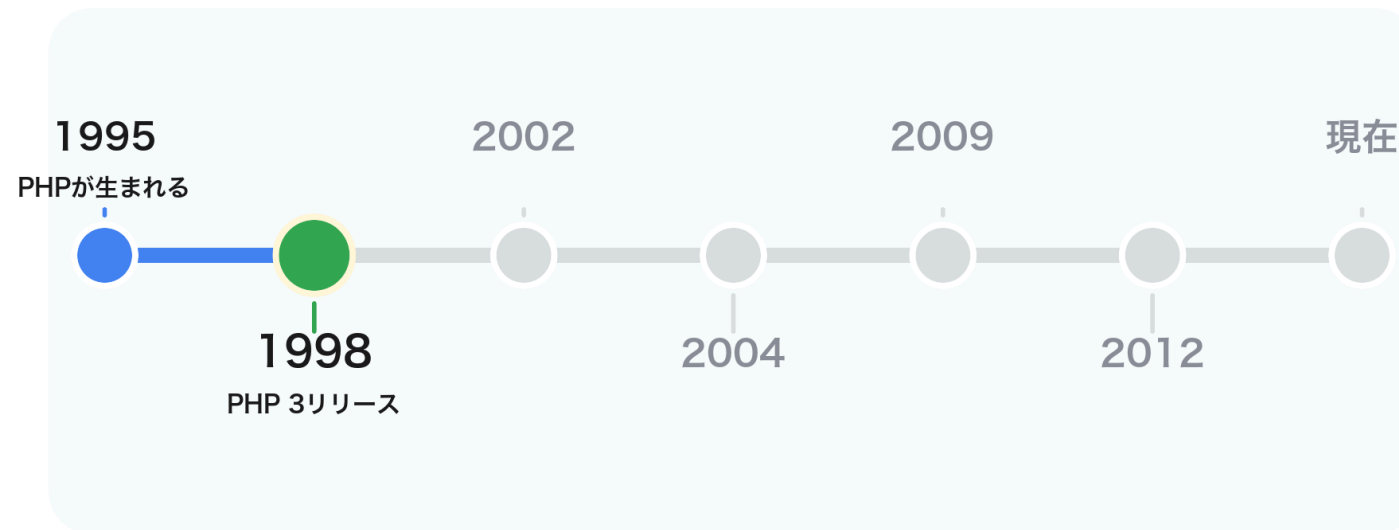
標準機能にも camelCase があります

```
$reflectionClass->getName();  
$reflectionMethod->getParameters();  
$dateTime->setTimezone($timezone);
```

正確にはグローバル関数ではなく、標準クラスのメソッド側です

1998 PHP 3リリース

今日歩くPHP命名史



関数文化 → OOP文化 → PSR → 現代の判断

PHP 3でPHPは今の姿に近づく



拡張される言語、増える関数

- 便利な機能がどんどん追加される
- Web開発で使う処理が関数として増える
- 命名の一貫性だけが主目的ではなかったはず

そして名前は、あとから揃えにくい

一度広く使われた関数名は 簡単には変えられません

動いているコードを壊さないため、名前も歴史として残っていきます

命名は「設計思想」だけでは決まらない

いつ作られたか

誰が作ったか

どの文化圏か

互換性を壊せるか

使われ方は何か

どの層のAPIか

2002 PEAR規約が登場

今日歩くPHP命名史



関数文化 → OOP文化 → PSR → 現代の判断

多分、かなり初期の共通規約...？

PEAR

PHP Extension and Application Repository
パッケージ配布とライブラリ文化の時代です

PEARの命名規約

対象	例
クラス	<code>Net_Finger</code> , <code>HTML_Upload_Error</code>
メソッド	<code>connect()</code> , <code>getData()</code>
定数	<code>DB_DATASOURCENAME</code>

PEARはちょっと面白い

- クラス名は `ClassName` で階層を表す
- メソッドはcamel系
- グローバル関数もPEAR規約ではcamel系を推奨

ここで「PHPの歴史、単純じゃないですね」となります

ここで大事なのは「ライブラリの層」

PHP標準関数

言語に最初から近い層

PEARパッケージ

共有ライブラリの層

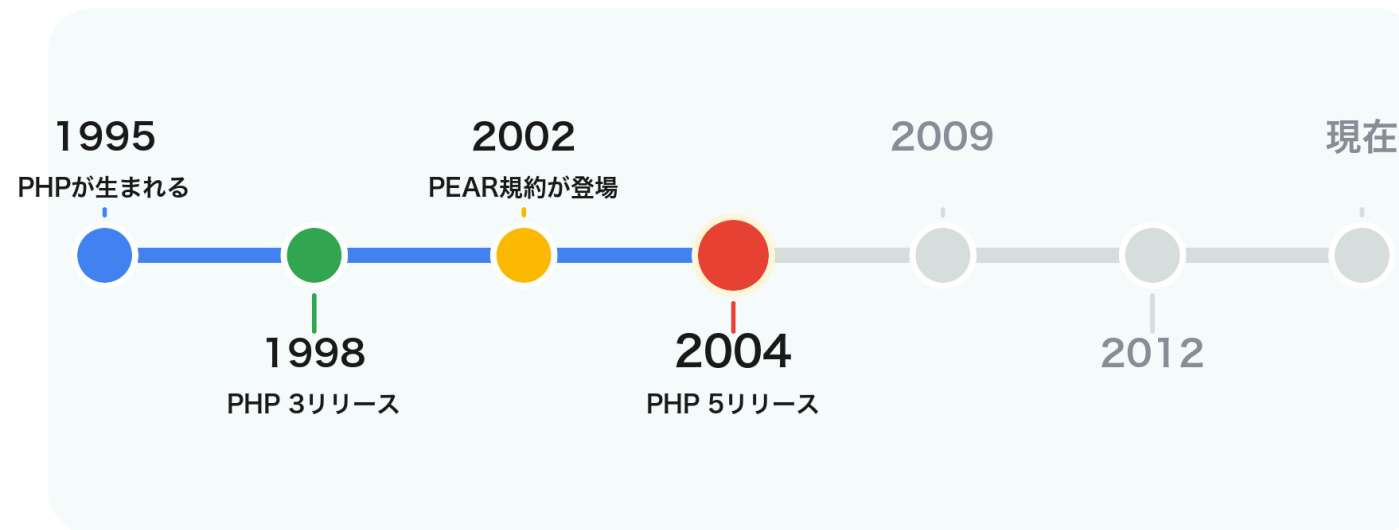
命名規則はすでに層で分かれていた

標準関数 ライブラリ クラス / メソッド

同じPHPでも、見ている層が違います

2004
PHP 5リリース

今日歩くPHP命名史



関数文化 → OOP文化 → PSR → 現代の判断

PHP 5でOOPが本格化します

Zend Engine 2.0

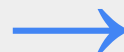
**新しい
オブジェクトモデル**

**クラスとメソッドを
書くPHPへ**

関数と呼ぶPHPから、メソッドを書くPHPへ

関数と呼ぶPHP

```
array_map($fn, $items);  
json_encode($payload);
```



メソッドを書くPHP

```
$request->getParsedBody();  
$response->getStatusCode();
```

メソッドにはcamelCaseの居場所があります

getParsedBody

getStatusCode

findById

setCreatedAt

hasPermission

isPublished

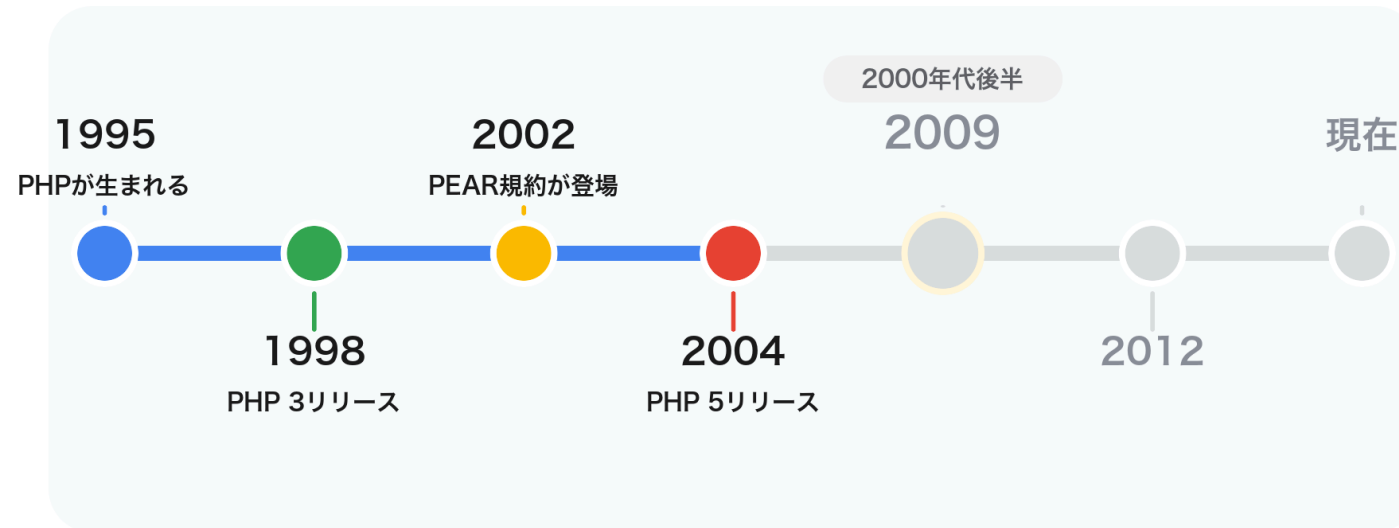
ここで文化が 分かれて見えてきます

標準関数の `snake_case`

OOPメソッドの `camelCase`

2000年代後半 フレームワーク文化

今日歩くPHP命名史



関数文化 → OOP文化 → PSR → 現代の判断

フレームワークが**PHP**の書き方を育てます

CakePHP

Symfony

Zend Framework

CodeIgniter

Doctrine

Solar / Lithium

多くのフレームワークでメソッドはcamelCaseへ

周辺	メソッド命名の印象
CakePHP	camelCase
Symfony	camelCase
Zend Framework	camelCase
Doctrine	camelCase
CodeIgniter	snake_case 寄り

ここは断言しません

一次資料として

「PSR-1がcamelCaseを選んだ理由」は

まだ見つけられていません

ここからは状況証拠からの考察です

仮説: もうみんなcamelCaseだったんじゃない？

**のちの共通規格は
突然生まれたわけではないのでは**

ここは状況証拠からの考察です

Symfony helperの余談

- Symfony 1系では基本的にcamelCase
- でもhelper関数はunderscore形式
- 理由は「helperは関数だから、PHP core functionに合わせた」

2つの文化はすでに共存していた

関数文化

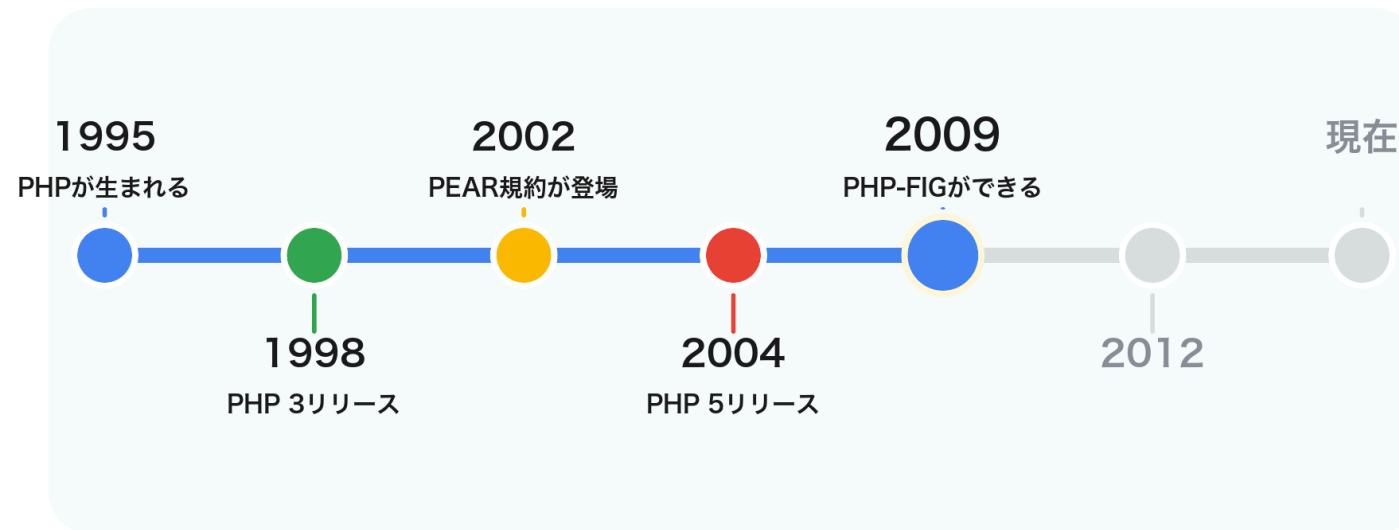
PHP標準関数に近い世界

OOP文化

フレームワークとメソッドの世界

2009
PHP-FIGができる

今日歩くPHP命名史



関数文化 → OOP文化 → PSR → 現代の判断

PHP-FIGとは

PHP Framework Interop Group

フレームワーク開発者たちが
相互運用のために集まったグループです

相互運用性がキーワード

- 共有されるPHPコード
- フレームワーク間で混ざりやすいコード
- ライブラリとして使いやすいコード

創設/初期メンバー周辺を見ると

代表的に関わったフレームワーク/ライブラリ

人	周辺プロジェクト	ここで見たいこと
Matthew Weier O'Phinney	Zend Framework	OOPフレームワーク
Fabien Potencier	Symfony	OOPフレームワーク
Paul M. Jones	Solar / Aura	ライブラリ設計
Jonathan Wage	Doctrine	ORM
Nate Abele	Lithium	フレームワーク

※ 厳密な「創設5名の完全対応表」は一次資料でまだ詰め切れていません。ここでは創設/初期メンバー周辺の代表的プロジェクトとして扱います。 48

その周辺の命名規則を見ると

フレームワーク/ライブラリとメソッド命名の傾向

周辺プロジェクト	メソッド命名の傾向	今回の読み
Zend Framework	camelCase	OOP API
Symfony	camelCase	OOP API
Solar / Aura	camelCase	ライブラリAPI
Doctrine	camelCase	ORM API
Lithium	camelCase	OOP API

※ ここでは主にユーザーランドOOPのメソッド命名として見ています。細部や関数名まで全部同じ、という話ではありません。

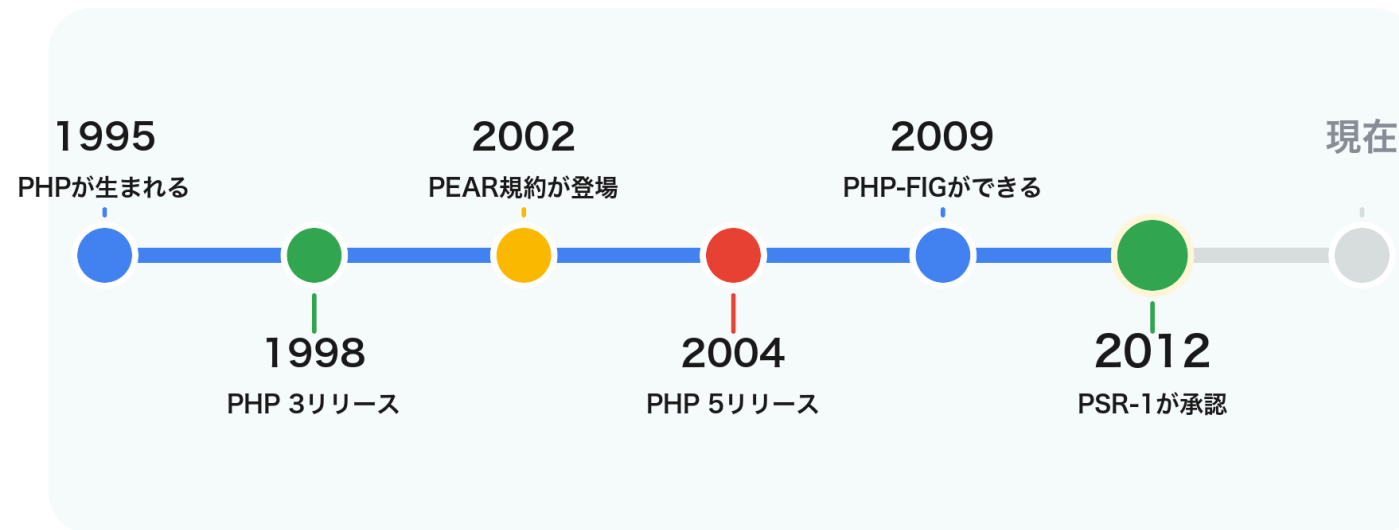
しょうもない仮説

会議室、もうだいぶ camelCaseの香りだった説

もちろん断言ではありません。
状況証拠からの「なんじゃね？（笑）」です

2012
PSR-1が承認

今日歩くPHP命名史



関数文化 → OOP文化 → PSR → 現代の判断

PSR-1: Basic Coding Standard

共有されるPHPコード間の 高い技術的相互運用性

PSR-1はこの目的で書かれています

PSR-1はPHP-FIGが作った規格

作成団体: PHP-FIG

Meta Documentでは
Editor: Paul M. Jones

さっき出てきた Solar / Aura 周辺の人です

フレームワーク開発者たちのグループが作った規格

公式仕様ではない

でも、PHPエコシステムでは事実上の標準として強いです
ぶっちゃけ他の規格もほぼおんなじこと言ってるし

PSR-1が命名について言っていること

対象	規則
クラス名	<code>StudlyCaps</code>
クラス定数	<code>UPPER_CASE_WITH_UNDERSCORES</code>
メソッド名	<code>camelCase</code>

プロパティ名はあえて決めていない

```
$StudlyCaps  
$camelCase  
$under_score
```

特定の推奨を避け、一貫性を求めています

PSR-1が言っていないこと

普通の関数名

変数名

標準関数の改名

じゃあ標準関数はなぜPSRっぽくないの？

- PSRより前から大量に存在していた
- 後から全部変えると互換性が壊れる
- そもそもPSRは標準関数に従うための規格ではない

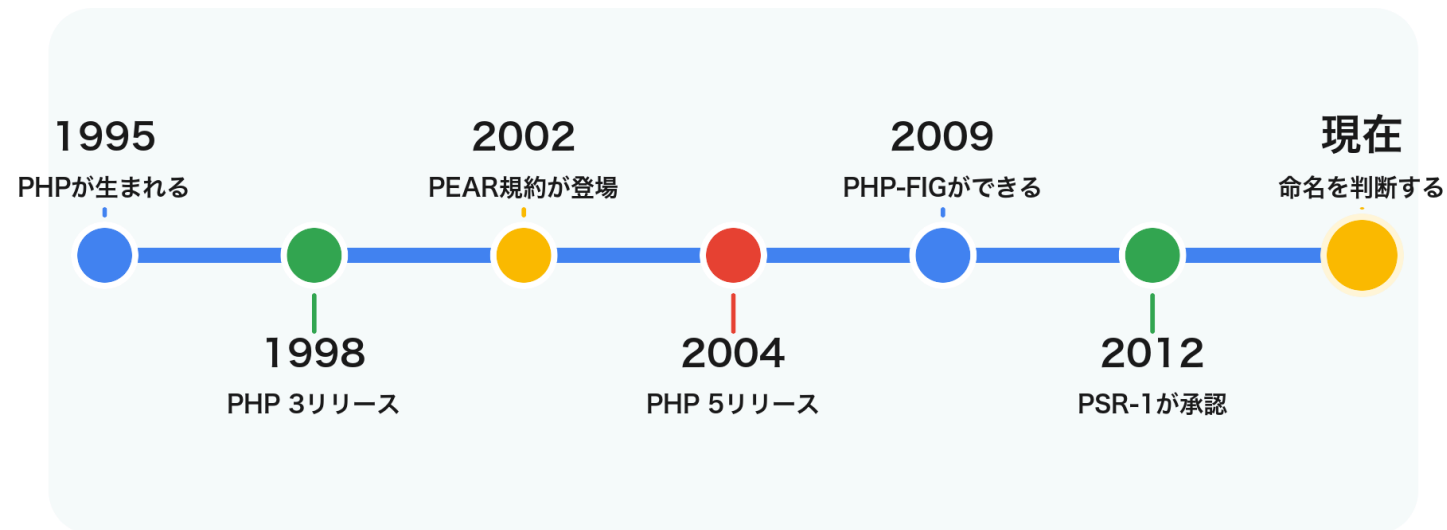
じゃあPSRはなぜ標準関数っぽくないの？

**当時のユーザーランドOOP文化を
固定したと見るのが自然そう**

もちろん、ここは断言ではなく考察です

現在
命名を判断する

今日歩くPHP命名史



関数文化 → OOP文化 → PSR → 現代の判断

PSRを知らなくてもPHPは書けます

動くコードは書けます

でも、チームで読むコードになると
命名のブレが急に痛くなります

それはそれとして 悶えることはあります

歴史やPSRを知らない人が書いたコードを読むと

命名で悶えることはあります

僕もそうだったので言うんですが

では、現代の私たちはどう判断する？

標準関数の文化

OOPの文化

PSRの規定

プロジェクトの一貫
性

FWの規約

互換性

普通の関数はsnake_case

対象

- グローバル関数
- ヘルパー関数
- 手続き的API

PHP標準関数の文化に寄せると自然

メソッドはcamelCase

対象

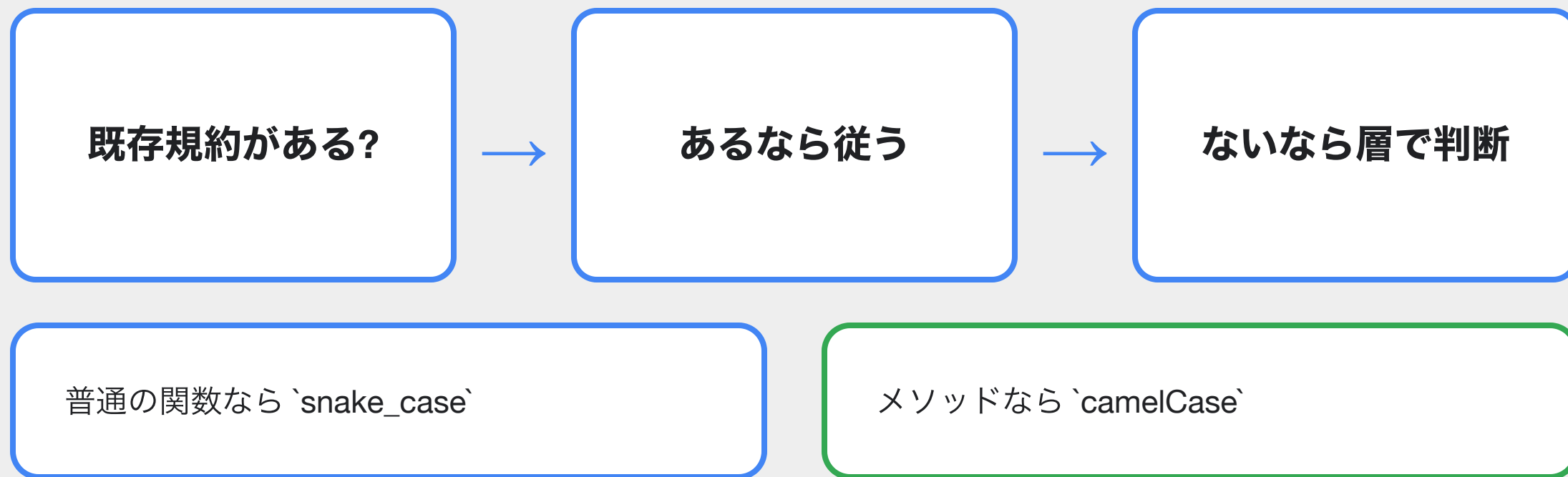
- クラスのメソッド
- サービスの操作
- オブジェクトの振る舞い

OOP PHP文化と、
その時点の最新規格に沿う

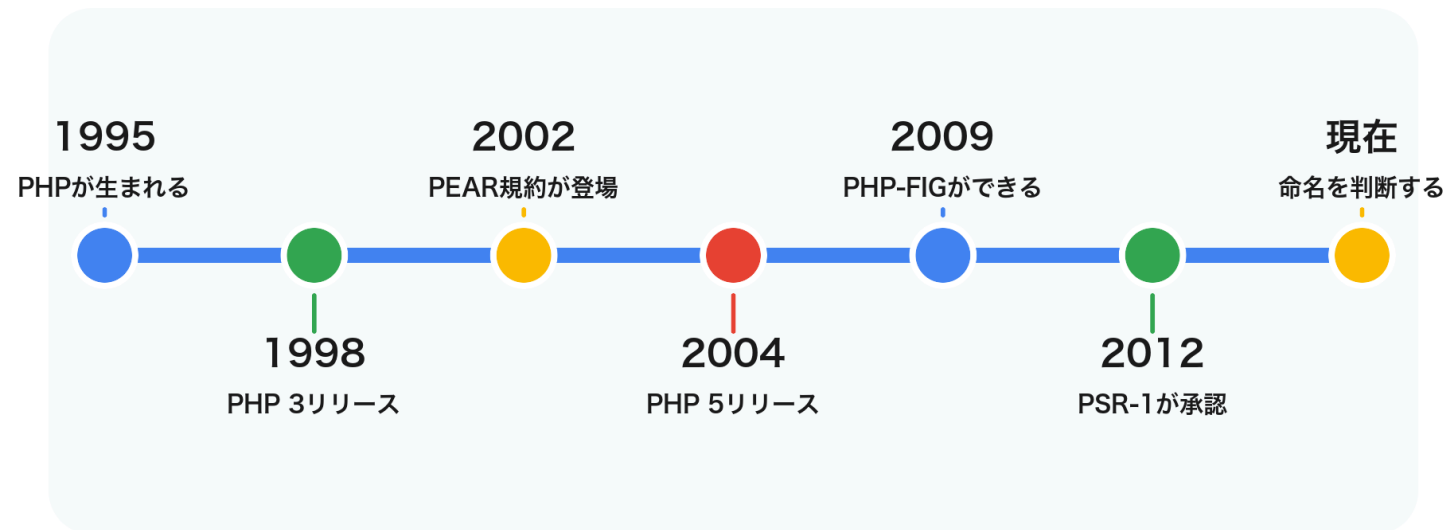
でも既存規約が最優先

- 既に `snake_case` メソッドで統一されているなら合わせる
- Laravel、Symfony、WordPressなどの周辺文化を尊重する
- 公開APIは互換性を壊さない

判断フローチャート



今日歩くPHP命名史



関数文化 → OOP文化 → PSR → 現代の判断

今日学んだこと

- PHPの命名規則は一枚岩ではない
- 関数文化とOOP文化は別々に育った
- PSRは言語仕様ではないが、事実上の共通規格として強い

私の結論

普通の関数は `snake_case`

メソッドは `camelCase`

絶対ルールではなく、歴史に沿った自然な落としどころ

出典

- PHP: History of PHP and related projects
- PHP-FIG: PSR-1 Basic Coding Standard
- PHP-FIG: PSR-1 Meta Document
- PEAR Manual: Naming Conventions
- PHP Manual: Userland naming rules
- Symfony 1.x documentation: helper naming discussion

記事版では各出典URLと補足を詳しく載せます

Thank you!

聞いてくれてありがとうございます

一緒にバンジー飛んでくれる人、探してます